

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester VI)
Artificial Intelligence Practical

Practical 7

Roll No.: T044	Name: SACHIN SINGH
Class: TYIT	Batch: 1
Date of Assignment: 1/07/2025	Date/Time of Submission: 1/07/2025

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

Code:

```
import pandas as pd
iris_df = pd.read_csv('/content/iris - iris - iris - iris.csv')
```

b. Perform basic preprocessing (handling missing values or scaling).

```
print('No missing values found in the Iris dataset.')

No missing values found in the Iris dataset.
```

Normalize Data (Min-Max Scaling)

Code:

```
from sklearn.preprocessing import MinMaxScaler
scaler_minmax = MinMaxScaler()
X_scaled_minmax = pd.DataFrame(scaler_minmax.fit_transform(X),
columns=X.columns)
print("Features after Min-Max Scaling:")
display(X_scaled_minmax.head())
```

Output:

```
Features after Min-Max Scaling:
   sepal_length  sepal_width  petal_length  petal_width
0      0.222222      0.625000      0.067797      0.041667
1      0.166667      0.416667      0.067797      0.041667
2      0.111111      0.500000      0.050847      0.041667
3      0.083333      0.458333      0.084746      0.041667
4      0.194444      0.666667      0.067797      0.041667
```

Standardize Data (Standard Scaling)

Code:

```
from sklearn.preprocessing import StandardScaler
scaler_standard = StandardScaler()
X_scaled_standard = pd.DataFrame(scaler_standard.fit_transform(X),
columns=X.columns)
print("Features after Standard Scaling:")
display(X_scaled_standard.head())
```

Output:

Features after Standard Scaling:				
	sepal_length	sepal_width	petal_length	petal_width
0	-0.915509	1.019971	-1.357737	-1.3357
1	-1.157560	-0.128082	-1.357737	-1.3357
2	-1.399610	0.331139	-1.414778	-1.3357
3	-1.520635	0.101529	-1.300696	-1.3357
4	-1.036535	1.249582	-1.357737	-1.3357

c. Split dataset into training and testing sets.

Code:

```
X = iris_df_processed.drop(columns=['species_encoded'])
y = iris_df_processed['species_encoded']
print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)

display(X.head())
display(y.head())
```

Output:

Features (X) shape: (147, 4) Target (y) shape: (147,)				
	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
species_encoded				
0	0			
1	0			
2	0			
3	0			
4	0			
dtype: int64				

d. Train a simple classifier.

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
X_train, X_test, y_train, y_test = train_test_split(X_scaled_minmax, y, test_size=0.3,
random_state=42)
print("X_train shape:", X_train.shape)
```

Output:

```
X_train shape: (102, 4)
X_test shape: (45, 4)
y_train shape: (102,)
y_test shape: (45,)
```

Code:

```
model = LogisticRegression(random_state=42, max_iter=200)
model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
```

Output:

```
Logistic Regression model trained successfully.
```

Code:

```
y_pred = model.predict(X_test)
print("Model Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
Model Accuracy: 0.9777777777777777

Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        17
     1           0.92        1.00        0.96        12
     2           1.00        0.94        0.97        16

   accuracy          0.98
  macro avg          0.97
weighted avg          0.98
```

e. Display training and testing accuracy.

Code:

```
y_train_pred = model.predict(X_train)
print("Model Training Accuracy:", accuracy_score(y_train, y_train_pred))
```

Output:

```
Model Training Accuracy: 0.9509803921568627
```

Q2 new dataset: Anime

a. Load dataset using Pandas.

Code:

```
import pandas as pd
df = pd.read_csv('/content/anime.csv')
print("First 5 rows of the dataset:")
display(df.head())
```

Output:

First 5 rows of the dataset:

	anime_id	name	genre	type	episodes	rating	members
0	32281	Kimi no Na wa.	Drama, Romance, School, Supernatural	Movie	1	9.37	200630
1	5114	Fullmetal Alchemist: Brotherhood	Action, Adventure, Drama, Fantasy, Magic, Mili...	TV	64	9.26	793665
2	28977	Gintama*	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.25	114262
3	9253	Steins;Gate	Sci-Fi, Thriller	TV	24	9.17	673572
4	9969	Gintama'	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.16	151266

b. Perform basic preprocessing (handling missing values or scaling).

Code:

```
print("\nDataFrame Info:")
display(df.info())
print("\nDescriptive Statistics:")
display(df.describe())
```

Output:

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12294 entries, 0 to 12293
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   anime_id    12294 non-null  int64
1   name        12294 non-null  object
2   genre       12232 non-null  object
3   type        12269 non-null  object
4   episodes    12294 non-null  object
5   rating      12064 non-null  float64
6   members     12294 non-null  int64
dtypes: float64(1), int64(2), object(4)
memory usage: 672.5+ KB
None

Descriptive Statistics:

```

	anime_id	rating	members
count	12294.000000	12064.000000	1.229400e+04
mean	14058.221653	6.473902	1.807134e+04
std	11455.294701	1.026746	5.482068e+04
min	1.000000	1.670000	5.000000e+00
25%	3484.250000	5.880000	2.250000e+02
50%	10260.500000	6.570000	1.550000e+03
75%	24794.500000	7.180000	9.437000e+03
max	34527.000000	10.000000	1.013917e+06

c. Split dataset into training and testing sets.

Code:

```
import numpy as np
print('Missing values before preprocessing:')
display(df.isnull().sum())

df.dropna(subset=['rating'], inplace=True)

df['episodes'] = pd.to_numeric(df['episodes'], errors='coerce')
df['episodes'].fillna(df['episodes'].mean(), inplace=True)

df['members'] = pd.to_numeric(df['members'], errors='coerce')
df['members'].fillna(df['members'].mean(), inplace=True)

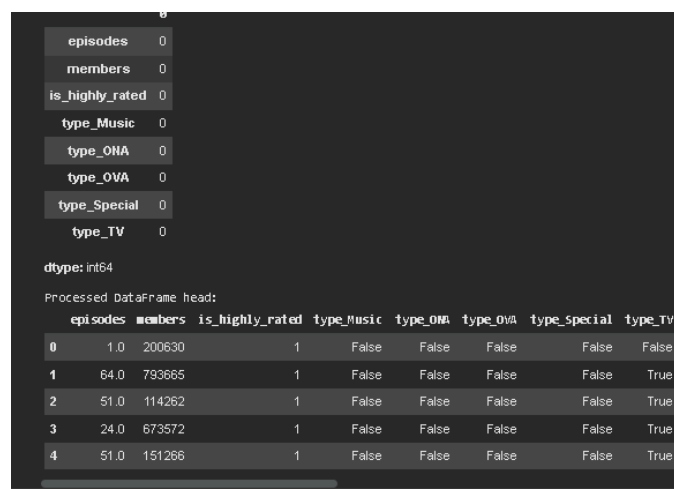
threshold = 7.0
df['is_highly_rated'] = (df['rating'] >= threshold).astype(int)

df_processed = df.drop(columns=['anime_id', 'name', 'genre', 'rating'])

df_processed = pd.get_dummies(df_processed, columns=['type'], drop_first=True)
print('\nMissing values after preprocessing:')
display(df_processed.isnull().sum())

print('\nProcessed DataFrame head:')
display(df_processed.head())
```

Output:



```
0
episodes    0
members     0
is_highly_rated  0
type_Music  0
type_OVA    0
type_OVA    0
type_Special 0
type_TV     0

dtype: int64

Processed DataFrame head:
```

	episodes	members	is_highly_rated	type_Music	type_OVA	type_OVA	type_Special	type_TV
0	1.0	200630	1	False	False	False	False	False
1	64.0	793665	1	False	False	False	False	True
2	51.0	114262	1	False	False	False	False	True
3	24.0	673572	1	False	False	False	False	True
4	51.0	151266	1	False	False	False	False	True

d. Train a simple classifier.

Code:

```
from sklearn.model_selection import train_test_split
X = df_processed.drop('is_highly_rated', axis=1)
y = df_processed['is_highly_rated']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

Output:

```
Shape of X_train: (9651, 7)
Shape of X_test: (2413, 7)
Shape of y_train: (9651,)
Shape of y_test: (2413,)
```

e. Display training and testing accuracy

Code:

```
from sklearn.tree import DecisionTreeClassifier

classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)
print("Decision Tree Classifier trained successfully.")
```

output:

```
Decision Tree Classifier trained successfully.
```

Comparing Datasets: anime.csv vs. iris - iris - iris - iris.csv

Code:

```
import pandas as pd

df_iris = pd.read_csv('/content/iris - iris - iris - iris.csv')

print('--- Iris Dataset ---')
print('\nFirst 5 rows:')
display(df_iris.head())

print('\nDataFrame Info:')
display(df_iris.info())

print('\nDescriptive Statistics:')
display(df_iris.describe())

print('\n--- Anime Dataset (Original before preprocessing) ---')

df_anime_original = pd.read_csv('/content/anime.csv')

print('\nFirst 5 rows:')
display(df_anime_original.head())

print('\nDataFrame Info:')
display(df_anime_original.info())

print('\nDescriptive Statistics:')
display(df_anime_original.describe())
```

Output:

```
First 5 rows:
  sepals_length  sepals_width  petals_length  petals_width  species
0             5.1           3.5           1.4           0.2    setosa
1             4.9           3.0           1.4           0.2    setosa
2             4.7           3.2           1.3           0.2    setosa
3             4.6           3.1           1.5           0.2    setosa
4             5.0           3.6           1.4           0.2    setosa

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   sepals_length  150 non-null    float64
1   sepals_width   150 non-null    float64
2   petals_length  150 non-null    float64
3   petals_width   150 non-null    float64
4   species        150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    sepal_length  150 non-null    float64
1    sepal_width   150 non-null    float64
2    petal_length  150 non-null    float64
3    petal_width   150 non-null    float64
4    species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

Descriptive Statistics:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

--- Anime Dataset (Original before preprocessing) ---

First 5 rows:

	anime_id	name	genre	type	episodes	rating	members
0	32281	Kimi no Na wa.	Drama, Romance, School, Supernatural	Movie	1	9.37	200630
1	5114	Fullmetal Alchemist: Brotherhood	Action, Adventure, Drama, Fantasy, Magic, Mil...	TV	64	9.26	793665
2	28977	Gintama*	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.25	114262
3	9253	Steins;Gate	Sci-Fi, Thriller	TV	24	9.17	673572
4	9969	Gintama'	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.16	151266

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12294 entries, 0 to 12293
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    anime_id    12294 non-null  int64
1    name        12294 non-null  object
2    genre       12232 non-null  object
3    type        12269 non-null  object
4    episodes    12294 non-null  object
5    rating      12064 non-null  float64
6    members     12294 non-null  int64
dtypes: float64(1), int64(2), object(4)
memory usage: 672.5+ KB
None
```